

DevGems — Full Technical Documentation

DevGems is a self-hosted web app for configuring database connections, taking **AES-256-GCM encrypted** backups (on-demand or scheduled), automatically **verifying** them, and **restoring** them into any target database — across PostgreSQL, MySQL/MariaDB, SQL Server, Oracle, MongoDB and SQLite.

This document describes the entire application: architecture, data model, security model, every workflow, the HTTP API, the command-line helpers, configuration, and day-to-day operations/troubleshooting.

Table of contents

1. [What it is](#)
 2. [Tech stack](#)
 3. [Directory layout](#)
 4. [Runtime architecture](#)
 5. [Data model \(SQLite\)](#)
 6. [Security model](#)
 - [Encryption at rest](#)
 - [Authentication \(key-based\)](#)
 - [Authorization \(RBAC\)](#)
 1. [Supported engines & required tools](#)
 2. [Client/server version matching \(important\)](#)
 3. [Core workflows](#)
 - [Connections](#)
 - [Taking a backup](#)
 - [Automatic verification](#)
 - [Scheduling](#)
 - [Retention](#)
 - [Download \(raw / decrypted\)](#)
 - [Restore](#)
 1. [HTTP API reference](#)
 2. [Environment variables](#)
 3. [Command-line helper scripts](#)
 4. [Installation & running](#)
 5. [Key management & rotation](#)
 6. [Operations & troubleshooting](#)
 7. [Extending: adding a new engine](#)
-

1. What it is

A single Next.js application that acts as a control panel for database backups. You register a **connection** (engine + host/port/db/user/password), then:

- **Back up** it manually or on a **cron schedule**. The dump tool's output is

streamed through an AES-256-GCM cipher directly to disk — plaintext never lands on the filesystem.

- **Verify** every backup automatically (integrity + completeness).
- **Restore** a backup into a **target** database (never the source), or **download**

the raw encrypted blob / a decrypted copy.

- Enforce **retention** (keep newest N, prune the rest).

All backup files and stored DB passwords are encrypted at rest with a key derived from a single master secret (`BACKUP_ENCRYPTION_KEY`). Access to the app is gated by **passwordless, key-based login** (ECDSA P-256) with **role-based permissions**.

2. Tech stack

Layer	Choice
Framework	Next.js 14 (App Router), React 18, TypeScript 5
Styling	Tailwind CSS 3 (dark glassmorphism, fully responsive)
Metadata store	SQLite via <code>better-sqlite3</code> (WAL mode)
Scheduling	<code>node-cron</code>
Crypto	Node built-in <code>crypto</code> (AES-256-GCM, scrypt); <code>@noble/curves</code> + <code>@noble/hashes</code> for ECDSA login
Dumps/restores	Native CLI tools per engine (<code>pg_dump</code> , <code>mysqldump</code> , ...) spawned as child processes

Native module note (`next.config.js`): `better-sqlite3` is kept external to the server bundle via `serverComponentsExternalPackages` .

3. Directory layout

```
backupDevgems/
├─ src/
│  ├─ app/
│  │  ├─ (app)/                # Authenticated area (shares layout with sidebar)
│  │  │  ├─ layout.tsx         # App shell: sidebar + full-width main
│  │  │  ├─ page.tsx           # Dashboard
│  │  │  ├─ backups/           # Backup history UI (details + restore tables)
│  │  │  ├─ connections/       # Connection list / new / edit
│  │  │  └─ users/             # Admin user management
│  │  ├─ api/                  # Route handlers (see API reference)
│  │  ├─ login/ , setup/       # Public auth pages
│  │  └─ layout.tsx , globals.css
│  ├─ components/             # Sidebar, ConnectionForm, shared UI, theme, etc.
│  └─ lib/
│     ├─ crypto.ts             # AES-256-GCM helpers (string + stream) + scrypt key
│     ├─ db.ts                 # SQLite schema, migrations, typed row accessors
│     ├─ engines.ts            # Per-engine dump/test command builders
│     ├─ backup.ts             # runBackup(): dump → encrypt → verify → retention
│     ├─ verify.ts             # Streaming integrity + completeness verification
│     ├─ restore.ts            # Streaming decrypt → load into target DB
│     ├─ scheduler.ts          # node-cron registration per connection
│     ├─ auth.ts / auth-edge.ts / auth-constants.ts # Sessions, guards
│     ├─ permissions.ts        # Permission keys + defaults
│     └─ serialize.ts, format.ts, user-dto.ts, webcrypto.ts
│  └─ middleware.ts            # Edge auth gate for all routes
│  └─ instrumentation.ts       # Starts the scheduler on server boot
├─ scripts/                    # Standalone Node/Bash operational helpers
│  ├─ verify-backup.cjs
│  ├─ restore-backup.cjs
│  └─ prune-backups.cjs
├─ dbbackup.sh                 # Interactive, version-aware backup helper
├─ install-pg15-client.sh       # Installs PostgreSQL 15 client alongside existing
├─ data/                       # (git-ignored) SQLite DB + encrypted backups
│  ├─ devgems.sqlite
│  └─ backups/*.enc
├─ .env.local                  # (git-ignored) secrets
└─ README.md / DOCUMENTATION.md
```

4. Runtime architecture

- **One Next.js process** serves the UI (React Server/Client Components) and the JSON API (route handlers under `src/app/api`). All DB-touching routes declare `runtime = "nodejs"`.
- `instrumentation.ts` runs once at server start and calls `initScheduler()`, which loads every connection that has a cron `schedule` and registers a `node-cron` task.
- **Backups/restores** are performed by spawning the engine's native CLI tool as a **child process** and streaming its stdout/stdin — so memory stays flat even for multi-GB databases.
- **State** lives entirely in `data/`: `devgems.sqlite` (metadata) and `data/backups/*.enc` (encrypted dumps). Nothing else is needed to run.

5. Data model (SQLite)

Defined and migrated in `src/lib/db.ts` (WAL mode, foreign keys on). Migrations are idempotent `ALTER TABLE ... ADD COLUMN` guards, so upgrading in place is safe.

connections

Column	Type	Notes
id	INTEGER PK	
name	TEXT	display name
engine	TEXT	<code>postgres</code> \ <code>mysql</code> \ <code>mariadb</code> \ <code>sqlserver</code> \ <code>oracle</code> \ <code>mongodb</code> \ <code>sqlite</code>
host, port	TEXT / INTEGER	port nullable
database	TEXT	DB name (for SQLite: file path)
username	TEXT	
password_enc	TEXT	encrypted (<code>iv:tag:cipher</code> hex), never plaintext
options	TEXT	extra CLI flags, space-separated
schedule	TEXT	cron expression, nullable
retention	INTEGER	keep newest N (0 = keep all)
created_at, updated_at	TEXT	ISO timestamps

backups

Column	Type	Notes
id	INTEGER PK	
connection_id	INTEGER FK	ON DELETE CASCADE
status	TEXT	<code>running</code> \ <code>success</code> \ <code>failed</code>
trigger	TEXT	<code>manual</code> \ <code>scheduled</code>
filename	TEXT	<code><name>-<timestamp>.<ext>.enc</code>
iv	TEXT	GCM IV (hex) needed to decrypt
auth_tag	TEXT	GCM auth tag (hex)
size_bytes	INTEGER	encrypted file size on disk
duration_ms	INTEGER	
error	TEXT	failure message
started_at, finished_at	TEXT	
verified	INTEGER	<code>null</code> = not checked, <code>1</code> = passed, <code>0</code> = failed
verify_error	TEXT	reason when verification fails

users

ECDSA public key, key id, fingerprint, `is_admin`, `permissions` (JSON), timestamps. (Legacy password columns are dropped on migration — auth is key-based only.)

sessions

Opaque random `token` (PK), `user_id`, `created_at`, `expires_at`, `user_agent`.

auth_challenges

Short-lived login nonces (id, username, nonce, expires_at) for challenge/response.

6. Security model

6.1 Encryption at rest

Implemented in `src/lib/crypto.ts`.

- **Key derivation:** `scryptSync(BACKUP_ENCRYPTION_KEY, "devgems.v1.salt", 32)` →

a 32-byte **AES-256** key. Derived in memory, **never persisted**. Requires the secret to be ≥ 8 chars.

- **Algorithm:** `aes-256-gcm`, 12-byte random IV per item, GCM auth tag for

tamper detection.

- **What's encrypted:**

1. **DB passwords** (`encryptString`) — stored as `iv:tag:cipher` hex in

`connections.password_enc`.

1. **Backup files** — the dump tool's stdout is piped through a GCM cipher

stream straight into `data/backups/<name>.enc`. The **IV + auth tag are stored in the `backups` row**, not in the file (the file is pure ciphertext).

- **Advantage:** stealing `data/` (backups *and* the metadata DB) yields nothing

usable without the key. GCM additionally guarantees integrity — any corruption or tampering fails decryption (this is what the "integrity VERIFIED" check relies on).

- **Trade-off:** losing `BACKUP_ENCRYPTION_KEY` makes all backups and stored passwords **permanently unrecoverable**. See [Key management](#).

6.2 Authentication (key-based)

No passwords. Uses ECDSA P-256 (`@noble/curves`).

- **First run** → `/setup`: browser generates a key pair, **downloads the private

key file **to the user's device, and registers only the public key**** on the server. After the first admin exists, `/setup` is locked.

- **Login** → `/login`: the browser requests a one-time **challenge (nonce)**, signs

it locally, and sends only the **signature**. The server verifies against the stored public key. **The private key never leaves the device / is never transmitted.**

- **Sessions:** a random opaque token stored in `sessions`, delivered as a

signed, HTTP-only, SameSite=Lax cookie (HMAC via `AUTH_SECRET`; `secure` in production).

- **Gate:** `src/middleware.ts` runs on every non-public route — unauthenticated API

calls get `401`, pages redirect to `/login`. Server pages/routes re-check the session against the DB (defense in depth). Login is rate-limited.

- **Recovery:** a lost key file cannot be recovered, but an admin can issue a new account/key.

6.3 Authorization (RBAC)

`src/lib/permissions.ts` defines permission keys; `requirePermission(key)` in `src/lib/auth.ts` guards each API route (returns `401` / `403`). The UI also hides controls the user can't use — but the **API is the source of truth**.

Permission	Allows
<code>viewBackups</code>	See connections & history; download backups
<code>runBackups</code>	Trigger backups, test connections, restore
<code>editConnections</code>	Create/modify connections (incl. schedule)
<code>deleteBackups</code>	Delete connections & backup files
<code>manageUsers</code>	Full user management (admins only)

Admins implicitly have all permissions. The last admin can't be demoted/deleted, and you can't delete your own account.

7. Supported engines & required tools

Per-engine behavior is in `src/lib/engines.ts` . The relevant CLI tool must be installed **on the host running DevGems** (not on the database server).

Engine	Default port	Dump tool	Test tool	Output ext
PostgreSQL	5432	<code>pg_dump</code>	<code>pg_isready</code>	<code>.sql</code> (plain)
MySQL	3306	<code>mysqldump</code>	<code>mysqladmin ping</code>	<code>.sql</code>
MariaDB	3306	<code>mysqldump</code>	<code>mysqladmin ping</code>	<code>.sql</code>
SQL Server	1433	<code>mssql-scripter</code>	(<code>dump --version</code>)	<code>.sql</code>
Oracle	1521	<code>sql</code> (SQLcl)	(<code>dump --version</code>)	<code>.sql</code>
MongoDB	27017	<code>mongodump</code>	<code>mongosh ping</code>	<code>.archive</code> (BSON)
SQLite	—	<code>sqlite3</code>	<code>sqlite3 SELECT 1</code>	<code>.sql</code>

Install examples (Debian/Ubuntu): `postgresql-client` , `mysql-client` / `mariadb-client` , `pip install mssql-scripter` , Oracle SQLcl download, `mongodb-database-tools` , `sqlite3` .

Credential handling: Postgres uses `PGPASSWORD` in the child env; MySQL/MariaDB use `MYSQL_PWD` ; these keep passwords off the visible process arg list where possible.

8. Client/server version matching (important)

For **PostgreSQL**, `pg_dump` 's major version **must be $\geq$ the server's** major version. A `pg_dump` 14 against a server 15 aborts with:

```
pg_dump: error: server version: 15.x; pg_dump version: 14.x
pg_dump: error: aborting because of server version mismatch
```

On Debian/Ubuntu, `/usr/bin/pg_dump` is the `pg_wrapper` , which automatically selects the **highest installed** client version for remote connections. So the fix is simply to install the matching client **alongside** the existing one — nothing on the database server changes:

```
bash install-pg15-client.sh      # adds PGDG repo + postgresql-client-15
```

The interactive `dbbackup.sh` automates this: it detects the server version and installs/uses a matching `pg_dump` . `psql` (used only to *read* the server version) is lenient and can talk to a newer server.

To **restore** a v15 dump you also need a v15+ **server** as the target (a v15 client alone is not enough).

9. Core workflows

9.1 Connections

Created/edited via the Connections UI (`ConnectionForm`) → `POST/PATCH /api/connections` . The password is encrypted before storage. **Test** (`POST /api/connections/[id]/test`) runs the engine's lightweight ping. Saving a connection with a `schedule` registers a cron task immediately.

9.2 Taking a backup

`runBackup(connection, trigger)` in `src/lib/backup.ts` :

1. Inserts a `running` backup row.
2. Builds the dump command (`buildDumpCommand`) and spawns the tool.
3. Pipes `tool stdout` → `AES-256-GCM cipher stream` → `data/backups/<name>.enc` .
4. **Zero-byte guard**: an empty dump (wrong path/creds) is treated as **failed**,

not silently "successful".

1. On success: computes the GCM tag, runs `verification`,

and records `success` , `filename` , `iv` , `auth_tag` , `size_bytes` , `duration_ms` , `verified` .

1. Applies `retention`.

Triggered by `POST /api/connections/[id]/backup` (manual, `maxDuration = 300`) or by the scheduler (scheduled).

9.3 Automatic verification

`src/lib/verify.ts` → `verifyEncryptedBackup(filePath, iv, tag, engine)` :

- **Streams** the file through the GCM decipher (authenticates the *entire* payload —

a bad tag/corruption fails), keeping only the first/last 8 KB in memory.

- Checks **engine completeness markers** on head/tail:
- Postgres: `PostgreSQL database dump` header and `... dump complete` footer

(footer missing ⇒ truncated).

- MySQL/MariaDB: `-- MySQL/MariaDB dump / Server version` header (footer soft).
- SQLite: `PRAGMA / BEGIN TRANSACTION / CREATE TABLE` .
- Binary engines (Mongo): integrity + non-empty only.
- **Constant memory**, so it verifies multi-GB backups reliably. The result is stored

as `verified` (1/0) + `verify_error` , and shown in the UI as ✓ **Verified** / ✗ **Unverified**.

9.4 Scheduling

`src/lib/scheduler.ts` uses `node-cron` . `initScheduler()` (called from `instrumentation.ts` at boot) registers a task per scheduled connection. On each fire it re-reads the latest connection row (so credential/setting changes apply) and calls `runBackup(..., "scheduled")` . Creating/updating/deleting a connection re-registers/unregisters its task.

9.5 Retention

`applyRetention(connection)` runs after every successful backup. If `retention > 0` , it keeps the newest N **successful** backups and deletes older ones (file + row). `retention = 0` keeps everything. Configure per connection in the edit form ("Retention (keep newest N, 0 = all)") or with `scripts/prune-backups.cjs` .

9.6 Download (raw / decrypted)

`GET /api/backups/[id]/download` streams the raw `.enc` blob. With `?decrypt=1` it pipes the file through the decrypt stream and returns the plaintext dump (filename without `.enc`). Streaming means large files don't need to fit in memory.

9.7 Restore

`src/lib/restore.ts` → `restoreBackup(row, target)` , exposed via `POST /api/backups/[id]/restore` and the **Restore** button in the UI.

- The **target is always caller-specified** (host/port/user/password/database) — it

is **never** the source connection, so a restore can't overwrite the live source.

- **Streaming:** the decrypted dump is piped straight into the client tool's stdin

(`psql` / `mysql` / `sqlite3`) — constant memory, handles multi-GB dumps.

- **PostgreSQL specifics:**
 - `ON_ERROR_STOP=1` — a broken restore fails loudly instead of half-applying.
 - **Recreate target** (optional): `DROP DATABASE IF EXISTS` + `CREATE DATABASE`

for a clean restore.

- **Auto-create roles** (optional, default on): the dump is scanned line-by-line

for roles referenced by `OWNER TO` / `GRANT` / `REVOKE` , and any missing ones are created (`DO $$ ... CREATE ROLE ... EXCEPTION WHEN duplicate_object ...$$`). This fixes `ERROR: role "... does not exist` .

- **MySQL/MariaDB:** optional `DROP/CREATE DATABASE` , then stream into `mysql <db>` .
- **SQLite:** stream into `sqlite3 <file>` .

10. HTTP API reference

All routes are JSON, `runtime = "nodejs"` , and guarded by `requirePermission` unless noted. Auth routes are public (used before login).

Auth

Method	Path	Purpose
POST	<code>/api/auth/setup</code>	First-run admin creation (public until an admin exists)
POST	<code>/api/auth/challenge</code>	Issue a login nonce
POST	<code>/api/auth/login</code>	Verify signature, create session
POST	<code>/api/auth/logout</code>	Destroy session
GET	<code>/api/auth/me</code>	Current user + permissions

Connections

Method	Path	Permission	Purpose
GET	<code>/api/connections</code>	viewBackups	List (passwords never returned)
POST	<code>/api/connections</code>	editConnections	Create
GET / PUT / DELETE	<code>/api/connections/[id]</code>	view / edit / delete	Read / update / remove
POST	<code>/api/connections/[id]/test</code>	runBackups	Test connectivity
POST	<code>/api/connections/[id]/backup</code>	runBackups	Run a backup now

Backups

Method	Path	Permission	Purpose
GET	<code>/api/backups</code>	viewBackups	History (optional <code>?connectionId=&limit=</code>)
DELETE	<code>/api/backups/[id]</code>	deleteBackups	Delete file + row
GET	<code>/api/backups/[id]/download</code>	viewBackups	Raw <code>.enc</code> (or <code>?decrypt=1</code> plaintext)
POST	<code>/api/backups/[id]/restore</code>	runBackups	Restore into a target DB

`restore` body: `{ host, port, user, password, database, create?, createRoles? }` .

Users (admin)

Method	Path	Permission	Purpose
GET/POST	<code>/api/users</code>	manageUsers	List / create (generate or paste public key)
PATCH/DELETE	<code>/api/users/[id]</code>	manageUsers	Update role/permissions / delete

11. Environment variables

Set in `.env.local` (dev) or the process environment (prod). `.env.local` and `data/` are git-ignored.

Var	Required	Description
<code>BACKUP_ENCRYPTION_KEY</code>	Yes	Master passphrase → AES-256 key. Long & random. Losing it = backups & stored passwords unrecoverable. Min 8 chars.
<code>AUTH_SECRET</code>	Recommended	HMAC secret for session cookies. If unset, derived from <code>BACKUP_ENCRYPTION_KEY</code> . Set before <code>npm run build</code>.
<code>DATA_DIR</code>	No	Location of SQLite DB + backups (default <code>./data</code>).
<code>NEXT_DIST_DIR</code>	No	Alternate Next build dir.

Generate strong values:

```
openssl rand -base64 48
# or
node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"
```

12. Command-line helper scripts

These live alongside the app and reuse the same crypto/format. Run from the project root. They read

`BACKUP_ENCRYPTION_KEY` / `DATA_DIR` from `.env.local`.

`dbbackup.sh` (interactive, version-aware)

Detects the source server version, ensures a matching local client exists (installing `postgresql-client-<major>` via PGDG if needed — **local only**, alongside existing clients), then dumps. Supports PostgreSQL and MySQL/MariaDB.

```
./dbbackup.sh
```

`install-pg15-client.sh`

Adds the PGDG apt repo and installs `postgresql-client-15` next to your existing client (nothing removed/downgraded; the DB server is untouched).

```
bash install-pg15-client.sh
```

`scripts/verify-backup.cjs`

Streaming verifier: authenticates a backup (GCM) and checks completeness; can persist the `verified` flag.

```
node scripts/verify-backup.cjs          # newest successful backup
node scripts/verify-backup.cjs --all    # every backup
node scripts/verify-backup.cjs --id 18
node scripts/verify-backup.cjs --file NAME.enc
node scripts/verify-backup.cjs --out dump.sql # also write decrypted plaintext
node scripts/verify-backup.cjs --all --write # persist verified flags to DB
```

scripts/restore-backup.cjs

CLI restore/decrypt. Interactive prompts for the target (or use flags). Safe: target is always specified, confirms before writing.

```
node scripts/restore-backup.cjs --out dump.sql          # decrypt only
node scripts/restore-backup.cjs --id 16 --to-host localhost \
  --to-port 5432 --to-user postgres --to-db restore_db --create --yes
```

Note: the CLI restore buffers the file and does **not** auto-create roles. For very large PostgreSQL dumps or dumps that reference missing roles, prefer the **UI Restore** (streaming + auto-role-creation), or pre-create the role(s) manually.

scripts/prune-backups.cjs

Apply "keep newest N" retention manually and/or persist it.

```
node scripts/prune-backups.cjs --keep 3          # dry-run preview
node scripts/prune-backups.cjs --keep 3 --apply  # delete extras
node scripts/prune-backups.cjs --keep 3 --set --apply # also persist retention=3
```

13. Installation & running

Requirements: Node 18+, plus the dump CLI for each engine you use (see §7).

```
npm install
cp .env.example .env.local # then set BACKUP_ENCRYPTION_KEY (and AUTH_SECRET)
npm run dev                # http://localhost:3000
```

First visit → **/setup** to create the admin (downloads your private key file). Then add a connection and take a backup.

Production:

```
# set AUTH_SECRET and BACKUP_ENCRYPTION_KEY in the environment first
npm run build
npm start
```

Put it behind HTTPS/a reverse proxy when exposed publicly so session cookies stay confidential. Persist **DATA_DIR** (it holds the DB and all backups).

14. Key management & rotation

BACKUP_ENCRYPTION_KEY protects both backup files and stored DB passwords. Because existing data is encrypted under the **current** key, you cannot simply change it without consequences.

- **Set it once**, before creating real backups. Store a copy in a password manager.
- **Rotating the key** (changing it later) makes existing **.enc** backups and saved

passwords undecryptable. Two safe paths:

1. **Clean slate:** set the new key → delete old backups → re-enter each

connection's password → take fresh backups.

1. **Re-encrypt (no data loss)**: decrypt each item with the old key and

re-encrypt with the new key (backup files + their `iv / auth_tag` , and each `password_enc`). This can be scripted as a one-pass `rotate-key.cjs` (old key → new key). *(Not shipped by default — ask if you want it.)*

15. Operations & troubleshooting

Symptom	Cause	Fix
aborting because of server version mismatch	<code>pg_dump</code> older than server	Install matching client (<code>install-pg15-client.sh</code>); <code>pg_wrapper</code> auto-selects it. See §8.
Restore fails: <code>Unexpected end of JSON input</code> in UI	Server returned no body (older code OOM'd on large dumps)	Fixed by streaming restore; ensure app is restarted.
Backup shows ✗ Unverified but is large & fine	Old verification loaded whole file into a string (V8 max string)	Fixed by streaming verify; re-verify with <code>node scripts/verify-backup.cjs --all --write .</code>
Restore: <code>role "X" does not exist</code>	Dump has <code>OWNER TO / GRANT</code> for a role absent on target	Keep Auto-create roles ticked (UI), or <code>CREATE ROLE "X"</code> ; manually.
Restore: <code>database ... already exists</code> / partial objects	Previous half-restore left objects	Tick Recreate target database (drops & recreates).
Backup marked failed , 0 bytes	Wrong DB name/path/creds → empty dump	Zero-byte guard flags it; fix connection and retry.
<code>"<tool>" is not installed</code>	Missing dump/restore CLI on the host	Install per §7.
Can't decrypt any backup	<code>BACKUP_ENCRYPTION_KEY</code> changed/lost	Restore the original key; without it, data is unrecoverable.
GCM authentication FAILED	File corrupted or wrong key	Check disk/file integrity; confirm the key matches.

Handy inspection:

```
# list connections + backups straight from the metadata DB
node -e 'const D=require("./node_modules/better-sqlite3");const db=new D("./data/devgems.sqlite",{readonly:true});
console.table(db.prepare("SELECT id,name,engine,retention,schedule FROM connections").all());
console.table(db.prepare("SELECT id,connection_id,status,verified,size_bytes,filename FROM backups ORDER BY id DESC
```

16. Extending: adding a new engine

1. Add the engine to `EngineId` and the `ENGINES` map in `src/lib/engines.ts`

(label, default port, tool, output `ext` , `network`).

1. Add a `case` in `buildDumpCommand` (how to dump to `stdout`) and, ideally,

`buildTestCommand` (a cheap connectivity ping).

1. If the dump is text with recognizable boundaries, add completeness markers in

`inspectDumpParts` (`src/lib/verify.ts`).

1. If it can be restored via a CLI reading stdin, add a branch in

`restoreBackup` (`src/lib/restore.ts`).

Everything else — encryption, scheduling, retention, history, RBAC, download — is engine-agnostic and works automatically.

Generated as living documentation for the DevGems app. Keep it in sync with `src/lib/*` when behavior changes.